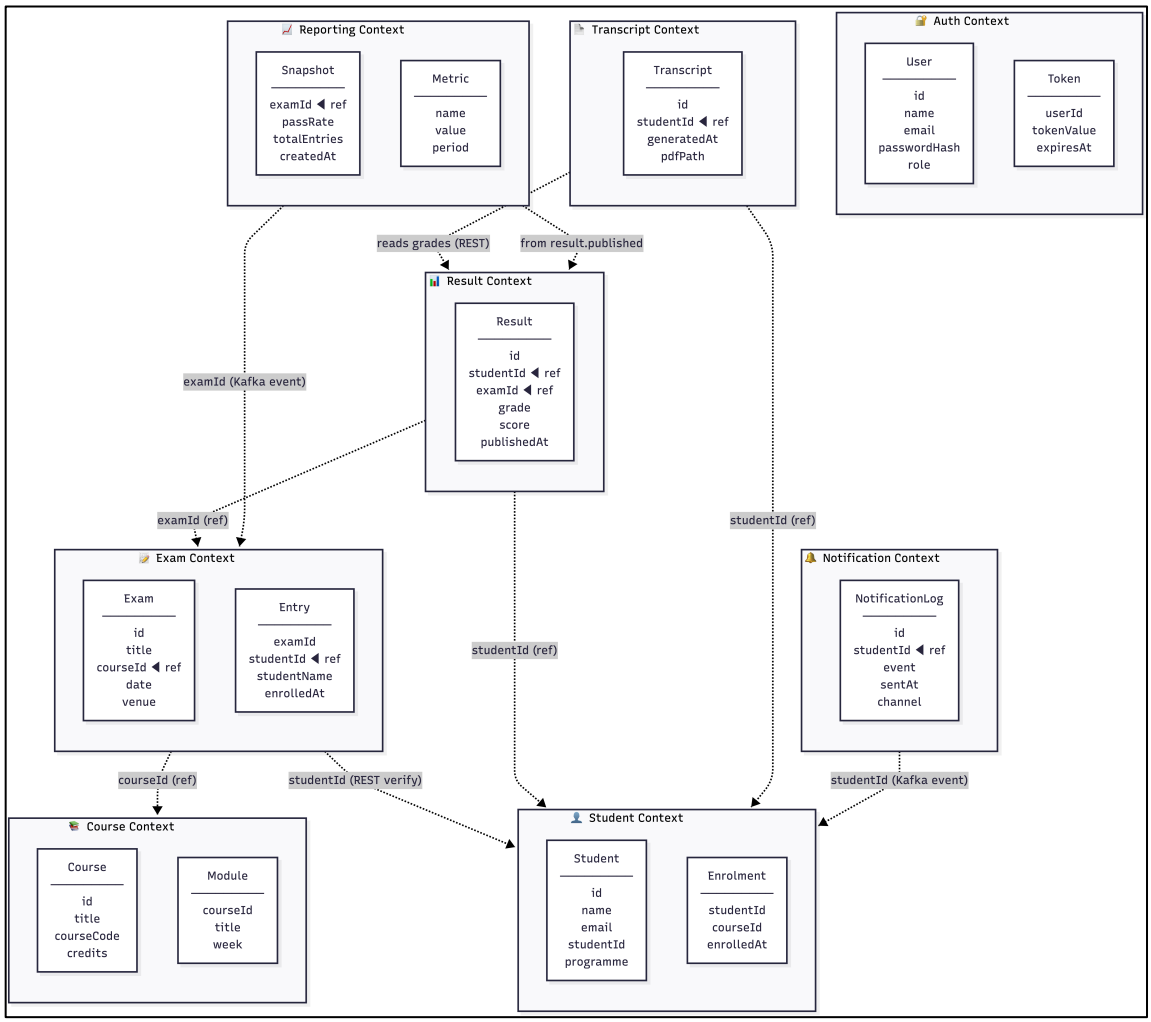
Bounded Context Diagram

University ERP · Advanced Software Engineering · UCSC

1. Purpose

A Bounded Context is a boundary within which a particular domain model is defined and applicable. This diagram maps each of the eight University ERP microservices to its own bounded context, showing the entities each service owns and the cross-boundary references that exist between contexts. No service reaches directly into another service's database — cross-boundary data access is performed only through REST APIs or Kafka event payloads carrying identifiers (IDs).

2. Diagram



3. Bounded Context Descriptions

3.1 Auth Context

Port	3007
Database	auth_db
Owned Entities	User (id, name, email, passwordHash, role) Token (userId, tokenValue, expiresAt)

Owns all identity and credential data. Responsible for issuing JWTs on login and managing password hashing with bcrypt. No other context stores credentials. All other services validate JWTs independently using the shared JWT_SECRET — they do not call Auth Service at runtime.

3.2 Student Context

Port	3001
Database	students_db
Owned Entities	Student (id, name, email, studentId, programme) Enrolment (studentId, courseId, enrolledAt)

The authoritative source for student identity in the system. When another context needs to verify a student, it calls this context's REST API and receives back the Student document. The studentId field is used as a shared identifier across contexts but the full Student document lives only here.

3.3 Course Context

Port	3002
Database	courses_db
Owned Entities	Course (id, title, courseCode, credits) Module (courseId, title, week)

Owns all course catalogue data. Other contexts reference courses by courseId only. The Exam context carries a courseId field on Exam documents as a reference, but never joins into courses_db directly.

3.4 Exam Context

Port	3003
Database	exams_db
Owned Entities	Exam (id, title, courseId ◀ref, date, venue) Entry (examId, studentId ◀ref, studentName, enrolledAt)

Manages the scheduling of exams and registration of students into exam sittings. Entry.studentId is a cross-boundary reference to the Student Context — before creating an Entry, Exam Service makes a synchronous REST call to Student Service to verify the student exists. studentName is denormalised onto Entry to avoid repeated cross-service lookups at read time.

3.5 Result Context

Port	3004
Database	results_db
Owned Entities	Result (id, studentId ◀ref, examId ◀ref, grade, score, publishedAt)

Records the outcome of each student's exam sitting. Both studentId and examId are cross-boundary references. When a result is published, this context emits a **result.published** Kafka event carrying studentId, examId, grade, and score — downstream contexts consume this event and maintain their own local copies of the data they need.

3.6 Transcript Context

Port	3005
Database	transcripts_db
Owned Entities	Transcript (id, studentId ◀ref, generatedAt, pdfPath)

Generates and stores PDF transcripts using pdfkit. Consumes **result.published** Kafka events to keep a local grade snapshot. When a transcript is requested on demand, it calls Result Service via REST to fetch the full grade list, generates the PDF, stores it, and records the metadata in transcripts_db.

3.7 Notification Context

Port	3006
Database	notif_db
Owned Entities	NotificationLog (id, studentId ◀ref, event, sentAt, channel)

A pure Kafka consumer context — it has no REST endpoints called by other services. Listens on **student.created**, **exam.registered**, and **result.published** topics. Sends email or SMS alerts and logs every notification to notif_db for audit purposes. studentId from the event payload is stored as a reference.

3.8 Reporting Context

Port	3008
Database	reports_db
Owned Entities	Snapshot (examId ◀ref, passRate, totalEntries, createdAt) Metric (name, value, period)

Aggregates system-wide metrics for the admin portal. Consumes **result.published** Kafka events and computes pass rates, grade distributions, and enrolment statistics. Stores pre-computed snapshots so admin queries are fast and do not put load on operational services.

4. Cross-Boundary References

The table below lists every cross-boundary reference shown as a dashed arrow in the diagram. All references are by ID only — no context holds a copy of another context's full entity.

From Context	To Context	Field / Mechanism	Purpose
Exam	Student	Entry.studentId — REST GET /api/students/:id	Verify student exists before exam enrolment
Exam	Course	Exam.courseId — reference only	Link exam to a course catalogue entry
Result	Student	Result.studentId — reference only	Identify which student the result belongs to
Result	Exam	Result.examId — reference only	Identify which exam sitting produced the result
Transcript	Student	Transcript.studentId — reference only	Identify the student the transcript is for

Transcript	Result	REST GET /api/results/:studentId	Fetch grade list when generating PDF on demand
Notification	Student	studentId in Kafka event payload	Identify recipient of notification
Reporting	Exam	examId in Kafka event payload	Attribute metric snapshot to an exam
Reporting	Result	**result.published** Kafka event	Source data for pass rate and grade distribution

5. Data Ownership Rules

- Each entity belongs to exactly one bounded context and is stored in only that context's database.
- A cross-boundary reference carries only the ID of the referenced entity, never the full object.
- The owning context is the single source of truth for its entities. Other contexts may cache a denormalised copy (e.g., Entry.studentName) for read performance, but must not write back to the owner's database.
- When an entity changes in its owning context, downstream contexts are updated via Kafka events, not by polling the owner's database or API.
- Shared identifiers (studentId, examId, courseId) are domain keys agreed upon at the system boundary — they are not database foreign keys enforced by a shared schema.

References: Service Identification Table, Architecture Design Document, Section 7 (Inter-Service Communication), Service Communication Diagram.